



Guia de Implementação: Módulo de Luzes Cozmo

Este documento detalha a criação do zero da funcionalidade de controle de LEDs, garantindo que o hardware e o software falem a mesma língua.



1. Estrutura de Pastas e Arquivos

Para manter o projeto organizado e modular, os arquivos de luzes foram alocados da seguinte forma:

- **js/blocks/cozmo_luzes.js**: Contém a definição visual (Blockly) e o gerador de lógica (JavaScript).
- **assets/**: Local onde as definições visuais e ícones de interface são armazenados para carregamento no **home.html**.



2. Criação do Bloco (Front-end & Lógica)

No arquivo **js/blocks/cozmo_luzes.js**, definimos como o bloco aparece para o aluno e como ele se transforma em código.

A. Definição Visual

O bloco utiliza um **Dropdown** para seleção de cores e um **Value Input** para o número de piscadas, garantindo flexibilidade.

JavaScript

```
Blockly.Blocks['cozmo_set_lights'] = {
  init: function() {
    this.appendDummyInput()
      .appendField("Cozmo: Luzes da Mochila")
      .appendField(new Blockly.FieldDropdown([
        ["vermelho", "red"],
        ["verde", "green"],
        ["azul", "blue"],
        ["branco", "white"],
        ["desligar", "off"]
      ]), "COLOR");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour("#cf63cf");
```

```
}  
};
```

B. O Gerador de Código

O "segredo" aqui é capturar o valor exato do campo `COLOR` e transformá-lo em uma chamada de função assíncrona que a nossa VM entenda.

```
JavaScript  
cozmoGenerator.forBlock['cozmo_set_lights'] = function(block) {  
  const color = block.getFieldValue('COLOR');  
  return `await ligarLuzes("${color}")`;   
};
```

3. A Ponte de Comunicação (VM)

No arquivo `executarCodigo.js`, criamos a função `ligarLuzes` dentro do objeto `cozmoActions`. Ela é a responsável por traduzir o desejo do bloco em um comando JSON para o Python.

Código da VM:

```
JavaScript  
ligarLuzes: async (cor) => {  
  const comando = { cmd: "set_lights", color: cor };  
  enviarParaPython(comando); // Envia via stdin para o servidor  
  
  // PAUSA CRÍTICA: O rádio do Cozmo precisa de tempo para processar o pacote  
  return new Promise(resolve => setTimeout(resolve, 300));  
},
```

4. O Servidor de Hardware (Python)

No `cozmo_server.py`, implementamos o receptor. O aprendizado técnico aqui foi a escolha da função `set_all_backpack_lights` para evitar conflitos de argumentos entre diferentes versões do PyCozmo.

```
Python  
elif cmd == "set_lights":  
  # 1. Sanitização do dado  
  color_name = str(data.get("color", "white")).lower()
```

2. Dicionário de Mapeamento de Hardware

```
colors = {  
    "red": pycozmo.lights.red_light,  
    "green": pycozmo.lights.green_light,  
    "blue": pycozmo.lights.blue_light,  
    "white": pycozmo.lights.white_light,  
    "off": pycozmo.lights.off_light  
}
```

3. Execução Física

```
selected_light = colors.get(color_name, pycozmo.lights.white_light)  
cli.set_all_backpack_lights(selected_light)
```

4. Feedback de Sucesso para o Log

```
time.sleep(0.1)  
log({"status": "ok", "cmd": "lights", "color": color_name})
```

5. Integração no **home.html** (Toolbox)

Para que o bloco apareça na interface, ele deve ser registrado no XML da Toolbox. O uso do **shadow block** no comando de piscar garante que o robô nunca receba um valor vazio.

XML

```
<category name="Luzes" colour="#cf63cf">  
    <block type="cozmo_set_lights">  
        <field name="COLOR">red</field>  
    </block>  
    <block type="cozmo_blink_lights">  
        <field name="COLOR">red</field>  
        <value name="TIMES">  
            <shadow type="math_number"><field name="NUM">3</field></shadow>  
        </value>  
    </block>  
</category>
```



Resumo do Trabalho Realizado (Checklist)

1. **Criação de Arquivo:** Desenvolvido `js/blocks/cozmo_luzes.js`.
2. **Injeção de Assets:** O script foi importado no cabeçalho do `home.html`.
3. **Tratamento de Erros de Argumento:** Resolvido o problema de `missing positional arguments` substituindo comandos específicos por `set_all_backpack_lights`.
4. **Persistência de Cor:** Implementado `time.sleep(0.1)` para evitar que o "heartbeat" do robô sobrescreva a cor escolhida pelo aluno.
5. **Modularização:** A lógica de piscar foi separada da luz fixa para permitir controle de fluxo (loops) via código.

Este documento agora reflete toda a engenharia por trás do módulo. Deseja que eu detalhe mais algum aspecto, como a criação dos ícones no CSS para esses blocos?